



**Intro to resilience modeling, simulation, and  
visualization in Python with fmdtools.**

**Author: Daniel Hulse**

**Version: 2.2.3**

# Overview

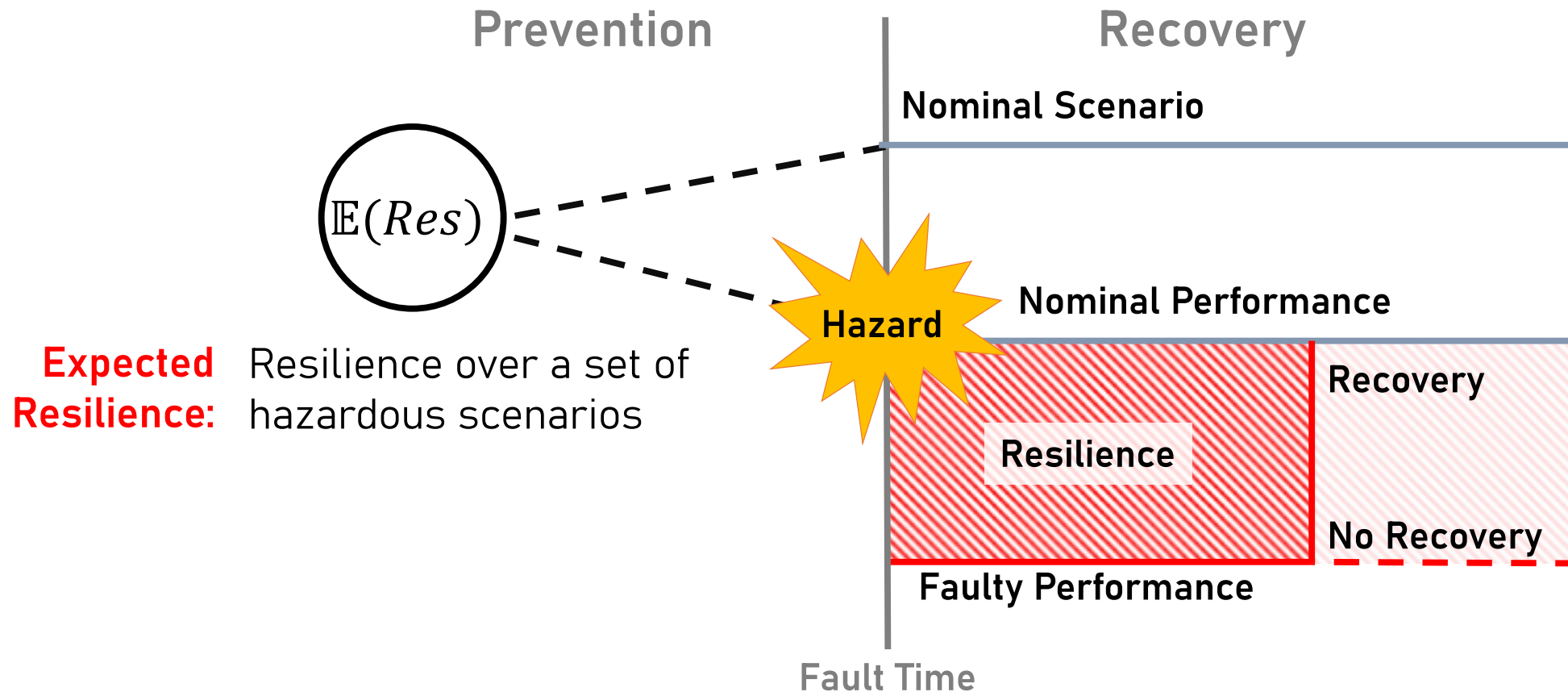
- **Overview of fmdtools**
  - Purpose
  - Project Structure
  - Common Classes/Functions
  - Basic Syntax
- **Coding Activity**
  - Example model: `examples/pump/ex_pump.py`
  - Workbook: `examples/pump/Tutorial_unfilled.ipynb`
    - Model Instantiation
    - Simulation
    - Visualization/Analysis

# Prerequisites

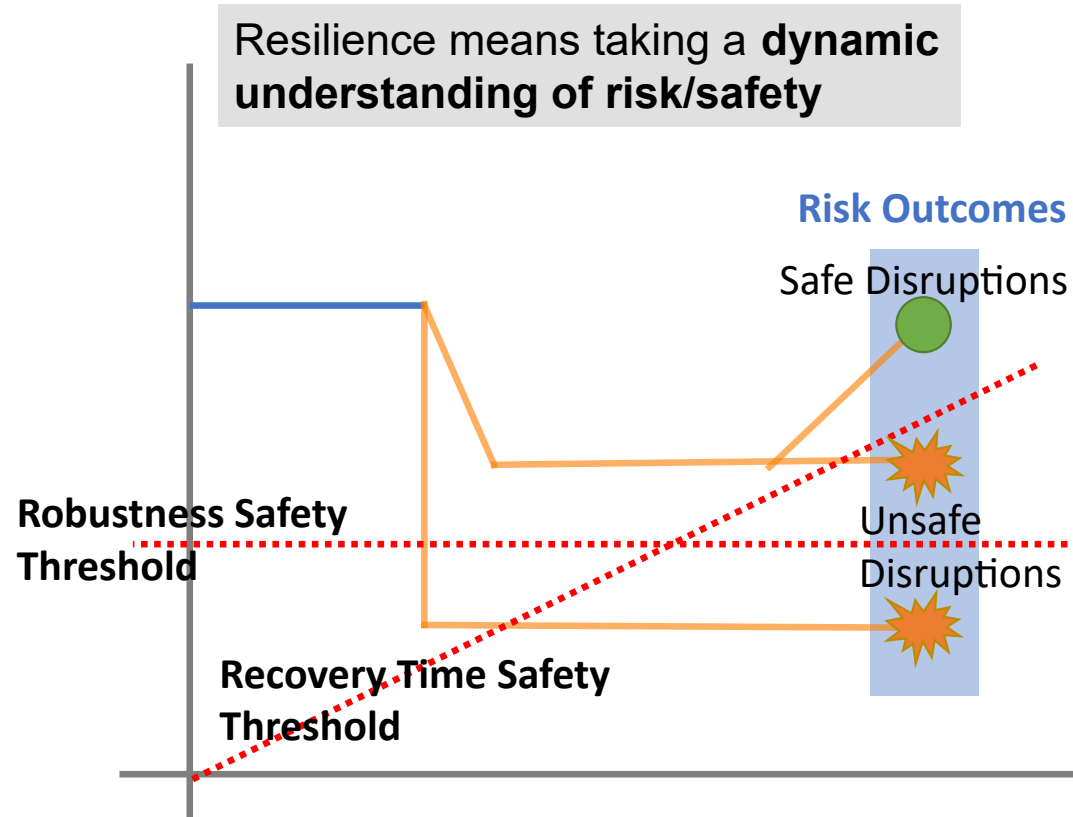
- Ideally, some pre-existing Python and Git knowledge
- Python distribution (anaconda or uv)
  - Ideally this is already set up!
  - Download/install from:
    - <https://www.anaconda.com/products/individual>
    - <https://github.com/astral-sh/uv>
- A git interface
  - [Github Desktop](#) (graphical git environment)
  - [git-scm](#) (stand-alone CLI)

# Motivation: Modeling System Resilience

Resilience means taking a dynamic understanding of risk and safety



# Why is Resilience Important?



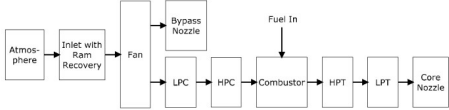
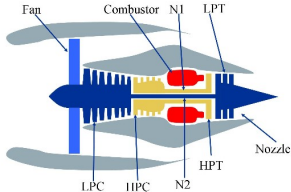
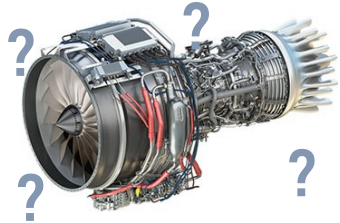
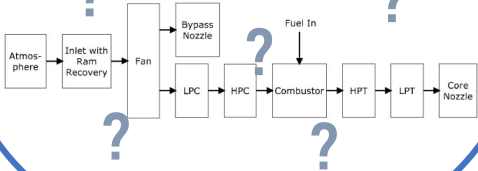
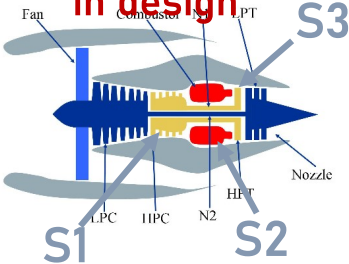
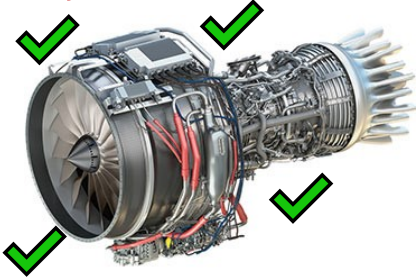
Considering resilience is important when our system has dynamic attributes, e.g.:

- **The system state changes over time**
  - (e.g., position, velocity, etc)
- **We can control this state**
  - (e.g., operators, autopilot)

Because we can use it to determine **how to control the system to a safe outcome** in unsafe circumstances and what **design/operational features** we need to enable this control

# Enabling a proactive design process - especially when we don't have data

Idea: the system should be **resilient-by-design**

|                  | Concept Design                                                                                                                 | Embodiment Design                                                                                                                         | Implementation                                                                                                               |
|------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Reactive Design  | <p><b>Choose concept</b></p>                  | <p><b>Design system</b></p>                            | <p><b>Retrofit for resilience</b></p>     |
| Proactive Design | <p><b>Establish resilience approach</b></p>  | <p><b>Integrate resilient features in design</b></p>  | <p><b>Verify resilient function</b></p>  |

# Why fmdtools? Possible Competitors:

- Uncertainty Quantification tools: (e.g. OpenCossan)
  - Does not incorporate fault modeling/propagation/visualization aspects
- MATLAB/modelica/etc. Fault Simulation tools
  - Rely on pre-existing model/software stack--Useful, but often difficult to hack/extend (**not open-source**)
- Safety Assessment tools: (e.g. Alyrica, Hip-Hops)
  - Focused on quantifying safety, not necessarily resilience
  - As a result, use **different model formalisms!**

# Why fmdtools? Pros:

- Highly Expressive, modular model representation.
  - faults from any component can propagate to any other connected component
  - highly-extensible code-based behavior representation
  - modularity enables **complex models** and modelling use-cases
- Research-oriented:
  - Written in/relies on the Python stack
  - Open source/free software
- Enables design:
  - Models can be parameterized and optimized!
  - Plug-and-play analyses and visualizations

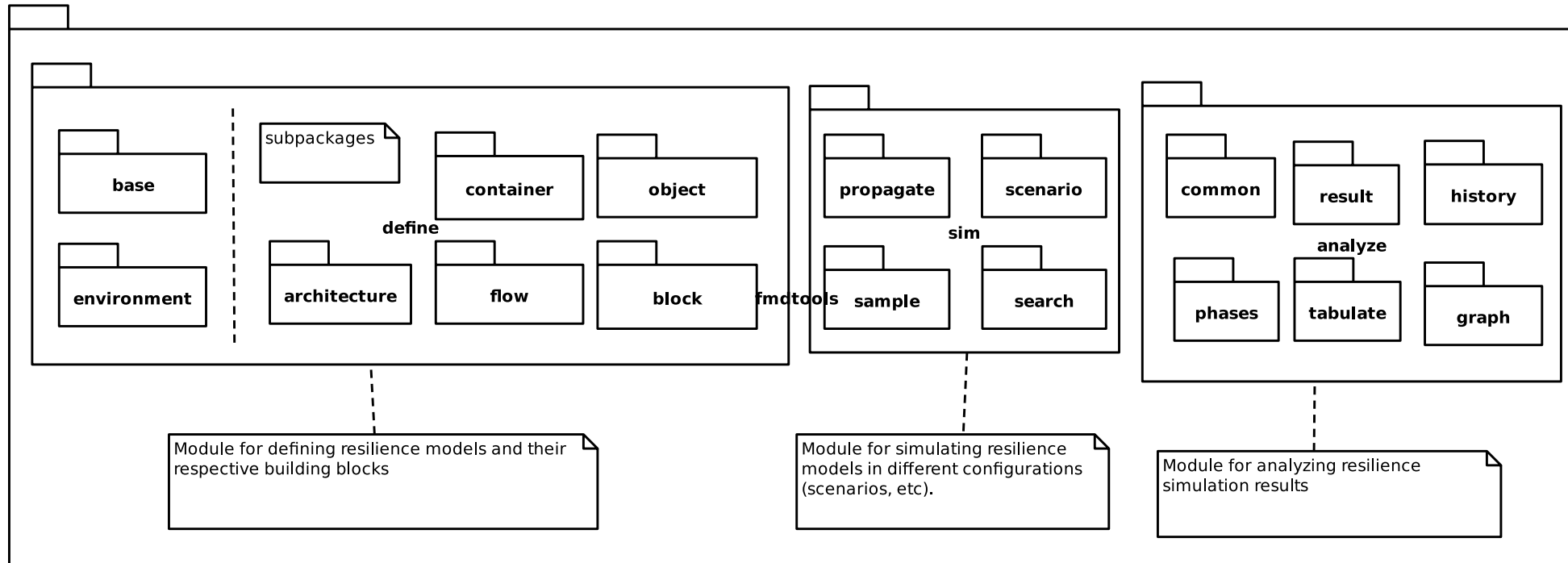


## Why **not** fmdtools? Cons:

- You already have a pre-existing system model
  - fmdtools models are built in fmdtools
  - if you have a simulink/modelica model, you may just want to use built-in tools
- You want to use this in production
  - fmdtools is Class E Software and thus mainly suitable for research (or, at least, we don't guarantee it)
  - Somewhat dynamic development history

# What is fmdtools? A Python package for design, simulation, and analysis of resilience.

pkg module organization



# What is fmdtools? Repo Structure

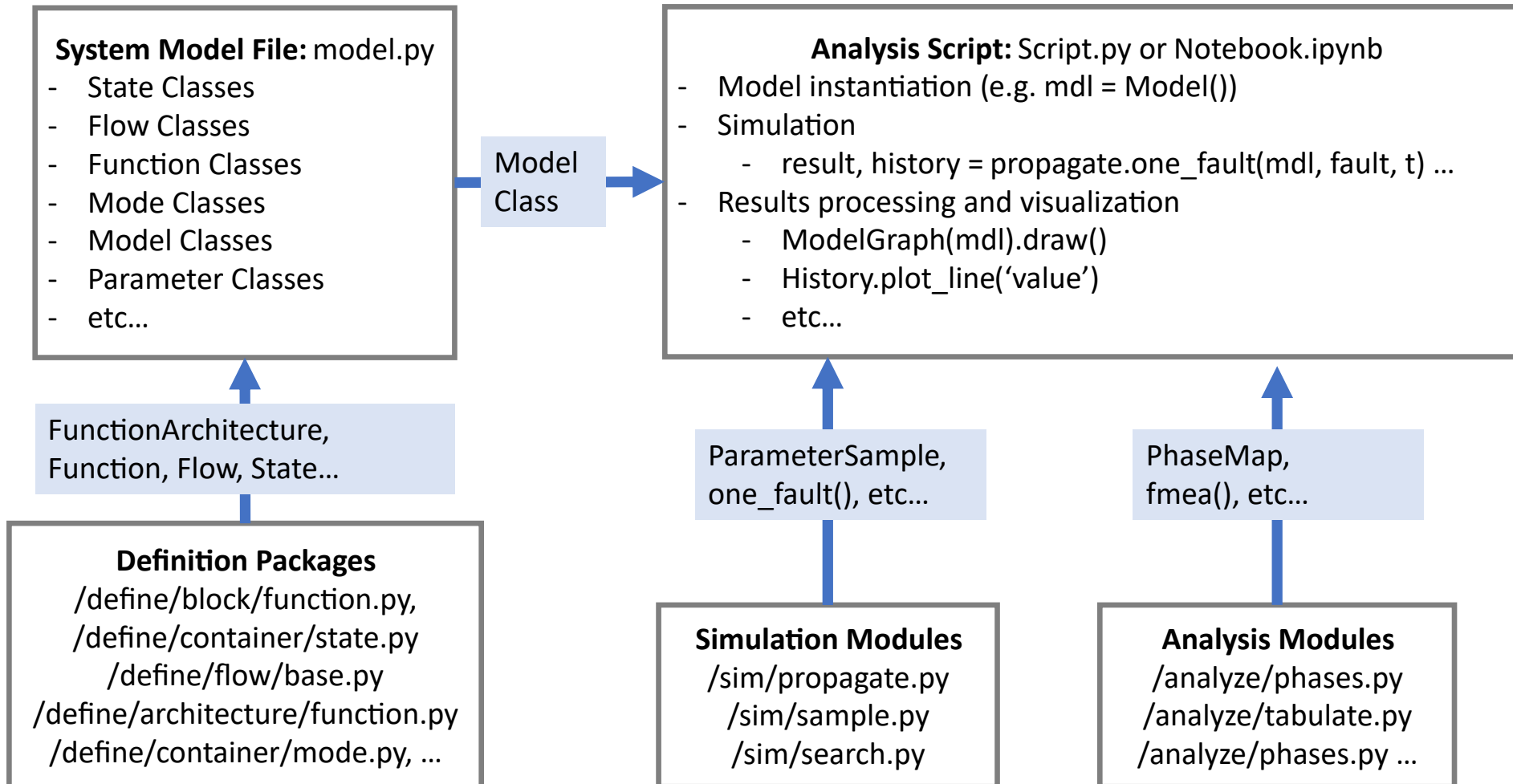
## Repository

- `/fmdtools` : installable package directory
- `/examples` : example models with demonstrative notebooks and tests
- `/docs` : [HTML Documentation](#) (source files at `/docs-source` )
- `/tests` : stand-alone tests (and testing rigs)
- Basic information: `README.md` , `CONTRIBUTORS.md` , `PUBLICATIONS.md` , `LICENSE` , `fmdtools_Individual_CLA.pdf` , etc.
- Config/test files: `requirements.txt` , `pyproject.toml` , `conf.py` , `index.rst` , etc.

# Activity: Download and Install fmdtools

- repo link: <https://github.com/nasa/fmdtools/>
- set up repo:
  - create `path/to/fmdtools` folder for repo
    - (usually in `/documents/GitHub` )
  - clone git into folder:
    - `git clone https://github.com/nasa/fmdtools.git`
    - can also use webpage
- package installation:
  - anaconda: Open Python from anaconda (e.g., open Spyder) and install with `pip install -e /path/to/fmdtools`
  - uv: run `uv pip install .` from fmdtools repository

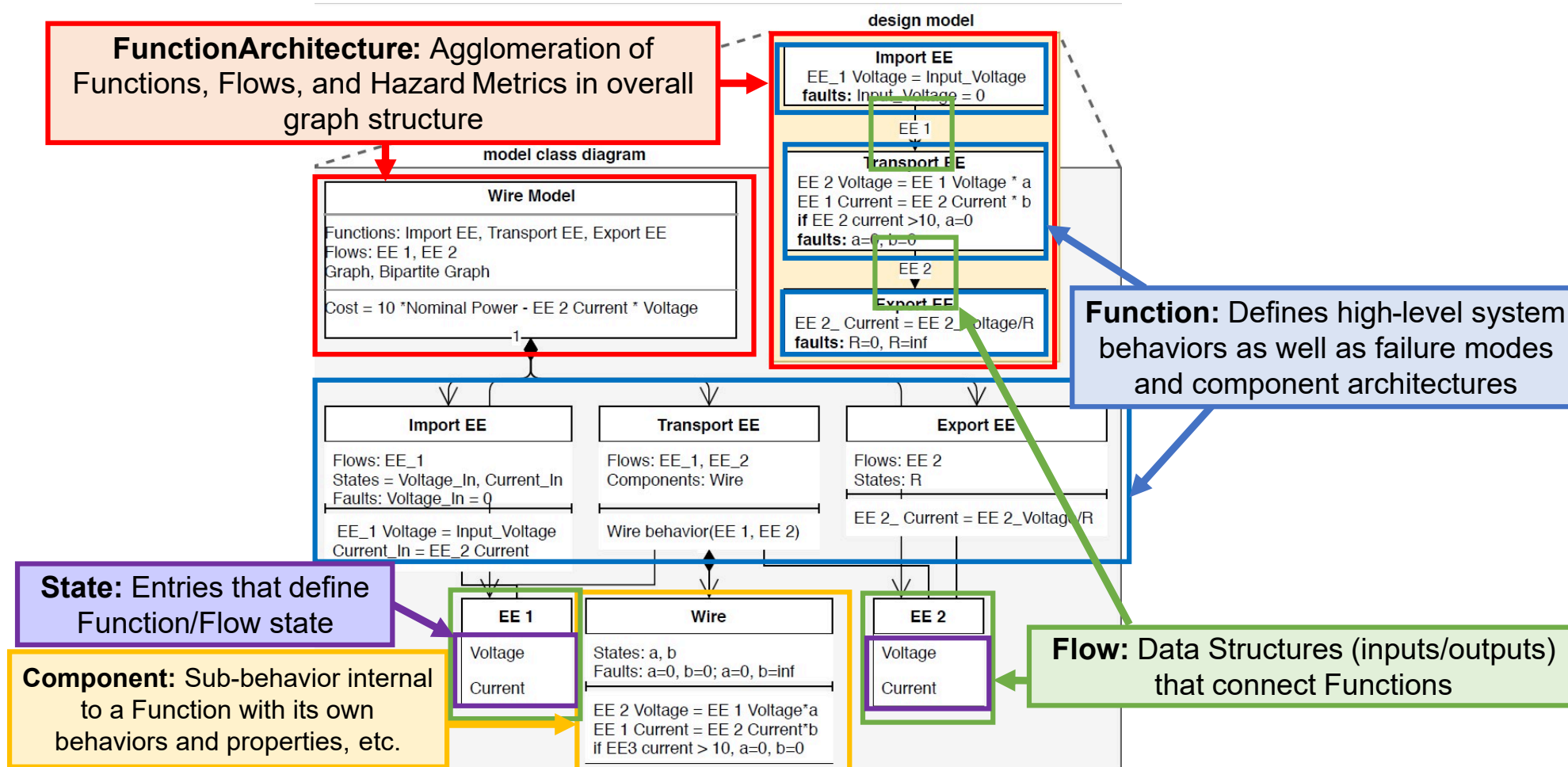
# Analysis Workflow/Structure



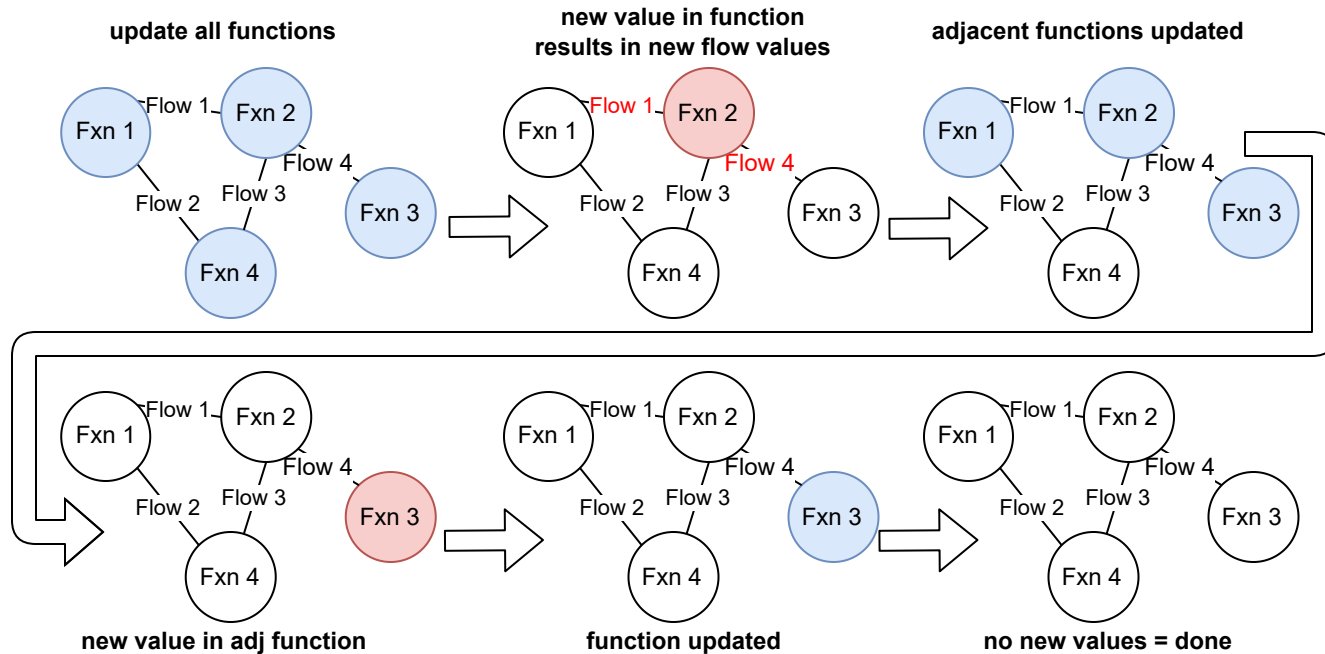
# Defining a Model

- What do we want out of a model?
  - What behaviors and how much fidelity do we need?
  - What **functions/components** and interactions make up the system?
    - One function or multiple functions?
    - Is it a **controlled system**? Are there multiple **agents**?
- What type of simulation do we want to run?
  - Single-timestep vs multi-timestep vs network
- What scenarios do we want to study and how?
  - **Failure modes** and faulty behaviors
  - **Disturbances** and changes in parameters
  - What are the possible effects of hazards and how bad are they?
    - By what **metrics**?

# Defining a Model



# Concept: Static Propagation

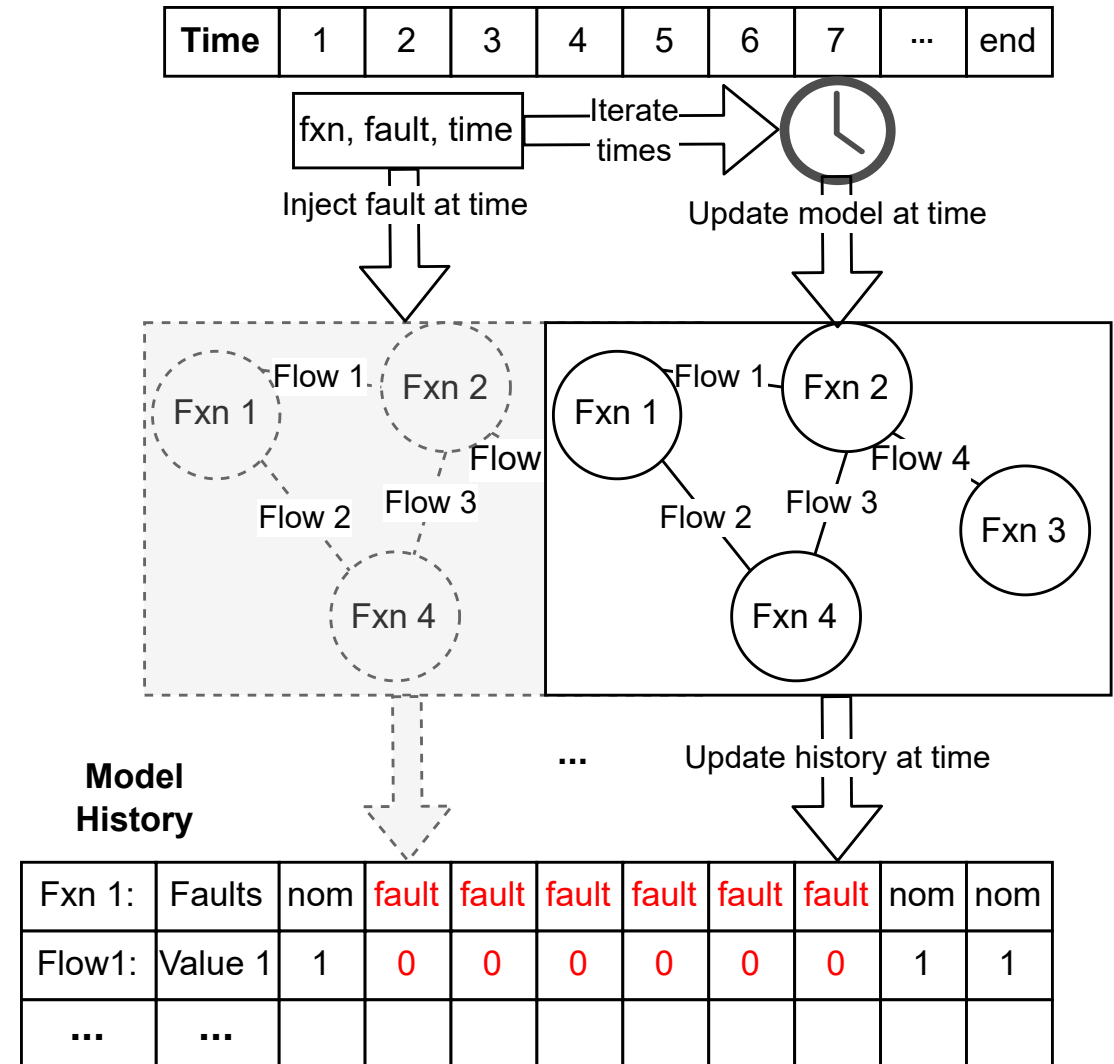


In a single timestep, functions with `static_behavior()` methods simulate until behaviors converge (i.e., no new state values)



# Concept: Propagation over Time

- Model increments (simulated + history updated) over each time-step until a **defined final time-step** or **specified indicator returns true**.
- Functions with `dynamic_behavior()` run once in defined order



# Python Concept: Classes and Object-Oriented Programming

Classes are the core of object-oriented programming. A class in python might look like:

```
class Hello(object):  
    a = 3 # <- Class variables define data used by all objects  
    def __init__(self, x=1, y=2): #<- This is a "dunder" method to instantiate the class  
        self.x = x  
        self.y = y  
  
    def sum(self):  
        return self.x + self.y + self.a
```

Classes are instantiated to form objects, which we can then use to manipulate data, e.g.:

```
>>> hi = Hello(x=2, y=2)  
>>> hi.sum()  
7
```

# Why classes and objects?

- Classes give you the ability to **group data and methods** together to satisfy an overall use-case
- Classes define the "template"--what should an object include, be able to do, how should it be instantiated, etc.
- **Inheritance**--you can create subclasses that creating new variants for a given system with specific properties (e.g., *Ferrari* might be a subclass of a *Car* class)
- **Principle of encapsulation:** Object-oriented programming give you control of what the user (or environment sees) versus what should be "internal"
- **Organization:** You always know what methods are supposed to act on what data
- **Independence:** New objects are independent of old objects, so, e.g., if you want a new sim while keeping the old one you can do that

# Python dataclasses (and similar)

fmdtools uses the recordclass package to define dataclasses:

<https://github.com/intellimath/recordclass>

```
from recordclass import dataobject
class Point(dataobject):
    x: int = 1 # <- Field for variable x
    y: int = 2 # <- Field for variable y
```

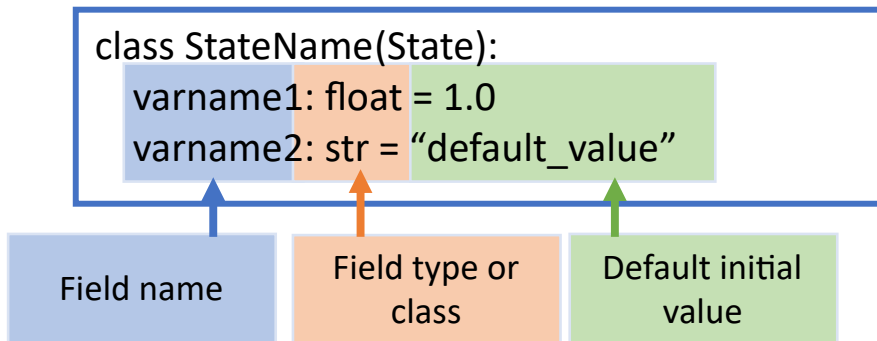
Dataclasses let you define the data as **fields** and then lets you directly instantiate the class based on the data, e.g.:

```
>>> p = Point(2, 3)
>>> p.x
2
```

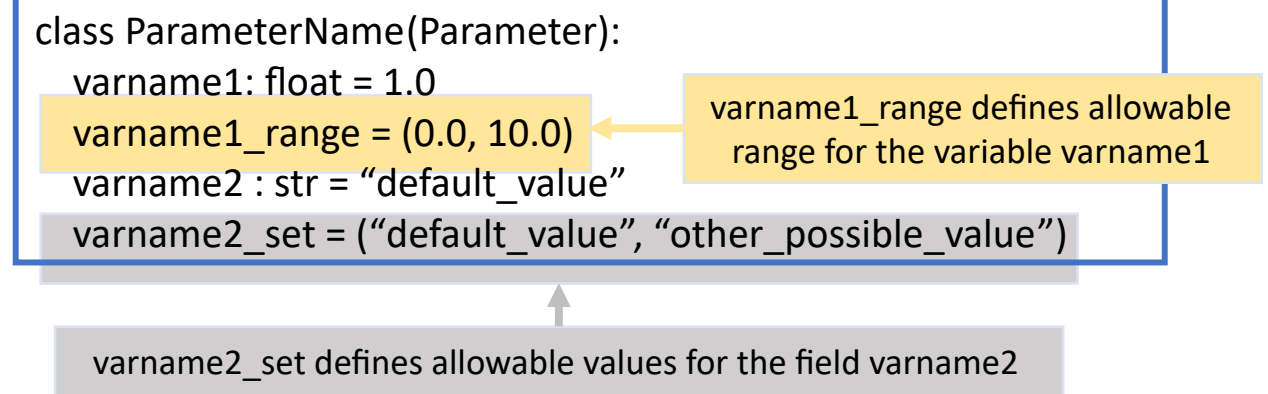
Dataclasses are used in fmdtools for **Containers**

# Containers - The building blocks of simulations

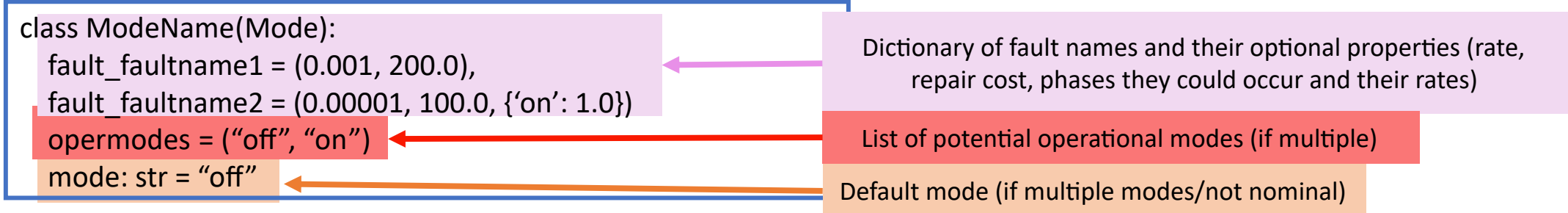
**State classes are used to represent variables (called fields) that change over time**



**Parameter classes are used to represent variables that don't change over time, with similar syntax to States**



**Mode classes are used to represent modes (faults and operational modes) that could occur in the system**



# Class aggregation in fmdtools BaseObject

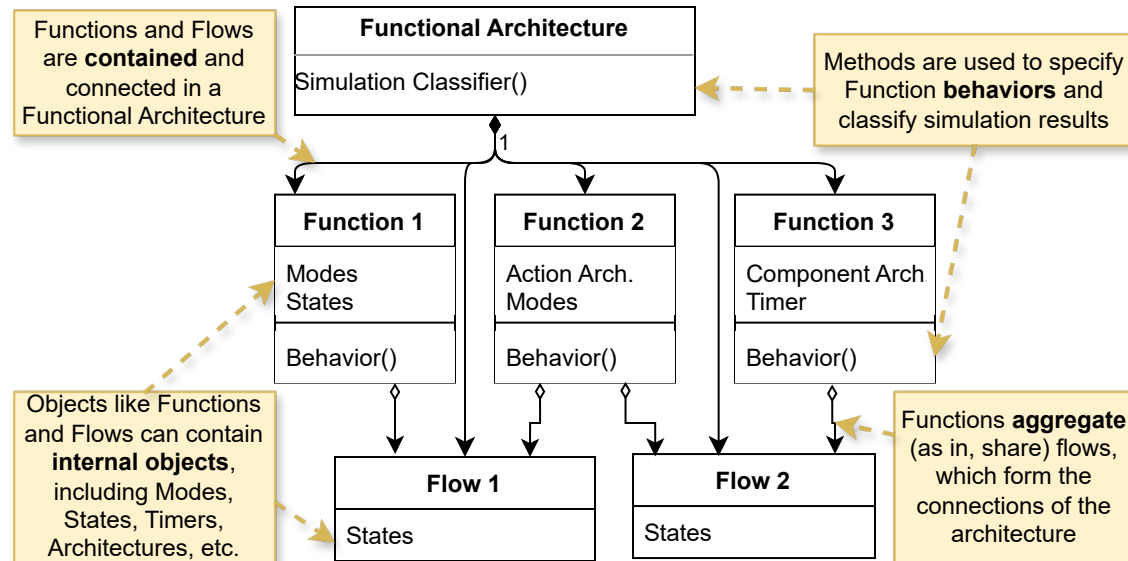
In fmdtools, **BaseObject** classes (Functions, Flows, etc.) aggregate Containers classes, allowing you to instantiate multiple at the same time, e.g., in the class:

```
class ExampleObject(BaseObject):  
    container_s = StateName
```

- `container_s` attaches the `StateName` class to the `ExampleObject` class as a container
- Our object class instantiates it to the variable `s`, e.g.:

```
>>> ex = ExampleObject()  
>>> ex.s.varname1  
1.0
```

# Main classes in fmdtools



**Containers** are used to represent properties of **Objects** (all classes below)

**Blocks** are the main behaviors of the system (e.g., Functions or Components)

**Flows** connect functions (and are thus a part of their definition)

**Architectures** are used to instantiate and connect Blocks with Flows

# Flow Code Template

```
class FlowName(Flow):
```

```
    container_s = StateName
```

```
    container_p = ParameterName
```

```
    default_track = ['s','m']
```

```
    def indicate_XXX(self):
```

```
        Conditional statement (e.g., self.s.state>threshold) which  
        is logged in the history and may be used to terminate  
        simulations
```

Specifies which container classes play specific Flow **roles** (e.g., s corresponds to the state role, p corresponds to a parameter role, m corresponds to the mode role, etc)

Specifies **what should be tracked** in the FxnBlock history (fxnname.h) by default. May be a dict ({role:value}), list ([role1, role2]), or string ("all","none", etc). Overwritten by track parameter at model instantiation.

These methods define Flow **indicators** and are called/tracked during simulation (in flow.i)

- Flows represent connections or shared variables between different functions. Think of them as Function inputs/outputs.
- Flows are build from container classes like states, along with their own methods/variables.



# Function Code Template

flow\_XXX is used to append a **flow** of given type that is named XXX to the function class. If the flow(s) has a different name outside the function (optional), flownames matches the external name to the internal name

Specifies **what should be tracked** in the FxnBlock history (fxnname.h) by default. May be a dict ({role:value}), list ([role1, role2]), or string ("all","none", etc). Overwritten by the track parameter during model instantiation.

These methods define the **behavior** of the FxnBlock and thus simulate at each time-step of the simulation.

These methods define FxnBlock **indicators** and are called/tracked during simulation (in fxnname.i)

This method defines the Result to be returned **when simulated individually**

```
class FunctionName(Function):
```

```
    container_s = FunctionState
    container_m = FunctionMode
    container_t = FunctionTime
```

Specifies which classes play specific FxnBlock **roles** (e.g., s corresponds to the state role, m corresponds to the mode role, etc)

```
    flow_flowname1 = FlowClass1
```

```
    flownames = {"outsideflowname": "flowname1"}
```

```
    default_sp = {'end_time': 100}
```

Default **keyword arguments for SimParam**. Only necessary when the functionblock will be simulated individually.

```
    default_track = ['s', 'm']
```

```
    def init_block(self, **kwargs):
        <e.g., self.s.x = 2.0>
```

**Optional** method to call to set up FxnBlock in ways not already defined by role initialization (e.g., attaching local MultiFlows or setting initial values for States from Parameter)

```
    def static_behavior(self):
```

Runs only in static propagation steps

```
    def dynamic_behavior(self):
```

Runs only in dynamic propagation steps

```
    def indicate_XXX(self):
```

Conditional statement (e.g., self.s.state > threshold) which is logged in the history and may be used to terminate simulations

```
    def classify(self, scen={}, hists={}, **kwargs):
```

Returns a Result dictionary (calculated at completion)

# Function Architecture Code Template

Default **keyword arguments for SimParam**.  
Defines max time of the simulation, along  
with phases, timestep, units, etc.

Method to **instantiate the model and define  
its structure**;

- **.add\_flow** is used to instantiate a flow
- **.add\_fxn** is used to instantiate a function  
and attach connected flows

These methods define Model **indicators**  
and are called/tracked during simulation  
(in modelname.i)

This method defines the Result to be  
returned by the model.

```
class ArchitectureName(FunctionArchitecture):
```

```
    container_p= ModelParam
```

```
    default_sp = {'end_time': 100}
```

```
    default_track = ["fxns", "flows"]
```

Points to a Parameter representing immutable model  
characteristics instantiated at the start of the simulation

Specifies **what should be tracked** in the Model  
history by default. May be a dict ({role:value}), list  
([role1, role2]), or string ("all","none", etc).  
Overwritten by the track option in propagate.

```
    def init_architecture(self, **kwargs):
```

```
        self.add_flow("flowname", FlowClass)
```

```
        ...
```

```
        self.add_fxn("functionname", FunctionClass, "flowname1", "flowname2")
```

```
        ...
```

```
    def indicate_XXX(self):
```

```
        Conditional statement
```

```
        (e.g., self.fxn["functionname"].s.state>threshold)
```

```
        which is logged in the history and may be used to terminate simulations
```

```
    def classify(self, scen={}, hists={}, **kwargs):
```

```
        Returns a Result dictionary (calculated at completion)
```

# Demo Model Activity: examples/pump/ex\_pump.py

Notice the definitions and structure:

- **States:** WaterStates , EEStates , SignalStates
- **Flows:** Water , EE , Signal
- **Functions:** ImportEE , ImportWater , ExportWater , MoveWater , ImportSignal
  - **Modes** (e.g., ImportEEMode , ImportSigMode )
    - Mode probability model
    - Actual modes in fm\_args entry
  - others attributes, e.g., Timer
- **Model:** Pump connects functions, flows, and defines end\_classification
- **Parameter:** PumpParam defines values we can change in the simulation

# Why do we structure models this way?

- **Enabling undirected propagation:** Behavior from any Block in an Architecture can propagate to anything else, which is important for capturing the full set of hazardous behavior
- **Composability and Parameterization:** OOP gives us the ability to create a number of co-existing model variants by varying parameters as well as creating sub-classes with different containers
- **Avoiding one big monolithic class:** We can document and unit test individual simulation pieces a lot easier than one giant class, increasing code quality

# More Resources for Model Definition

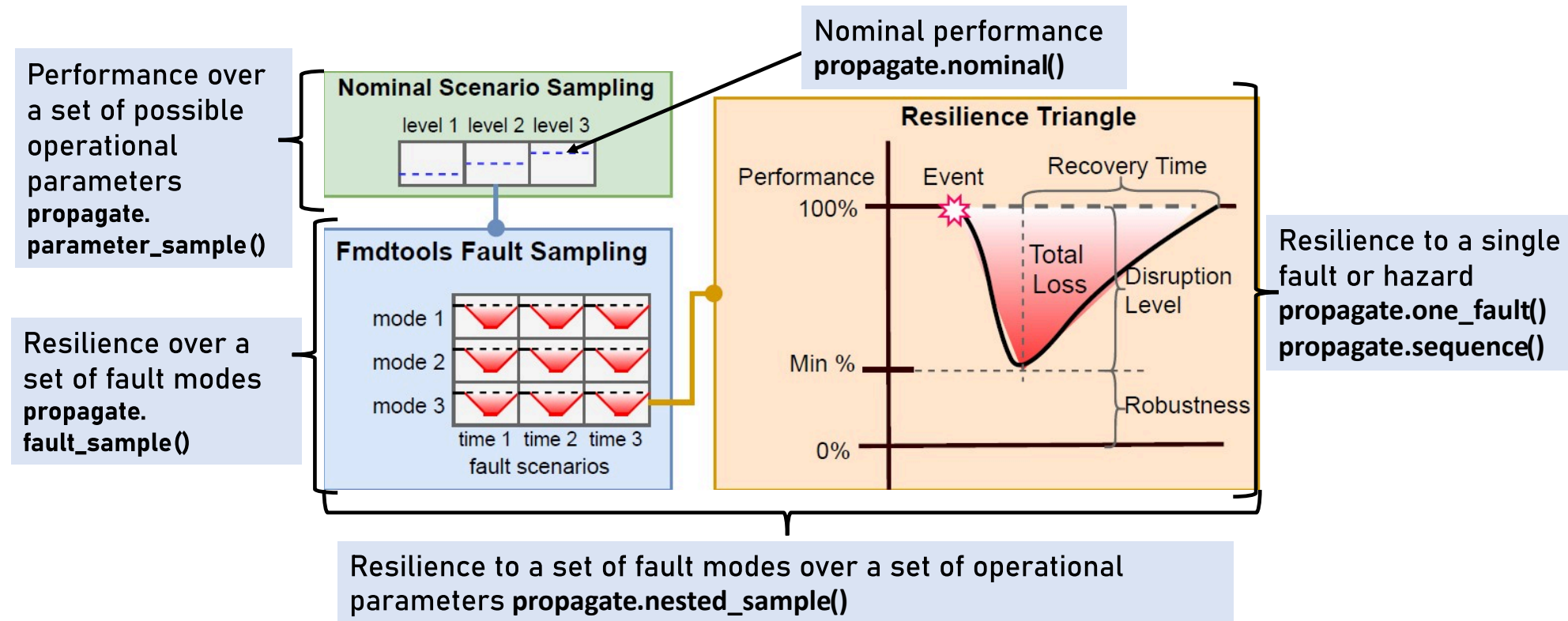
- Note the docs for model definition are in <https://nasa.github.io/fmdtools/docs-source/fmdtools.define.html>
- Other examples also can be helpful:  
<https://nasa.github.io/fmdtools/examples/Examples.html>

# Notebook Activity:

Open `/examples/pump/Tutorial_unfilled.ipynb` :

- Instantiate the model
  - `mdl = Pump()`
- Explore structure
  - Try different parameters!
  - Change things!  
What does the model directory look like?
  - `dir(mdl)`
- simulate to given time with `mdl(time)`

# Simulation Concepts: Types of Simulations



For more info on syntax/arguments, see documentation for [fmdtools.sim.propagate](#).

# Simulation Concepts: Sampling Approaches

These classes define **multi-run simulations** which can be used to quantify uncertain performance/resiliences:

- **SampleApproach/FaultSample**: Which faults to sample and when
  - Relies on **mode** information encoded in the model
  - Simulated using `propagate.fault_sample()`
- **ParameterSample**: Nominal parameters or random seeds to sample
  - Can be simulated in `propagate.parameter_sample()`
  - Can be simulated in conjunction with faults using `propagate.nested_sample`

See docs for: `fmdtools.sim.fault_sample`



# Simulation Concepts: Things to Consider

**Static/Dynamic propagation:** How function states propagate to each other in a single time-step and multiple time-steps?

**Stochastic Propagation:** Whether and how stochastic states are instantiated over time

- e.g. do we run with the “default” values of parameters, or do we sample from a random number generator?

**Breadth of Scenarios:** How hazards are represented as discrete scenarios to simulate

- What set of joint faults do we use? How many times are sampled?
- Operational scenarios and joint operational/fault scenarios

# Activity: Simulate the Model

Run fault propagation methods:

- `propagate.nominal()` , `propagate.one_fault()` , `propagate.fault_sample()`

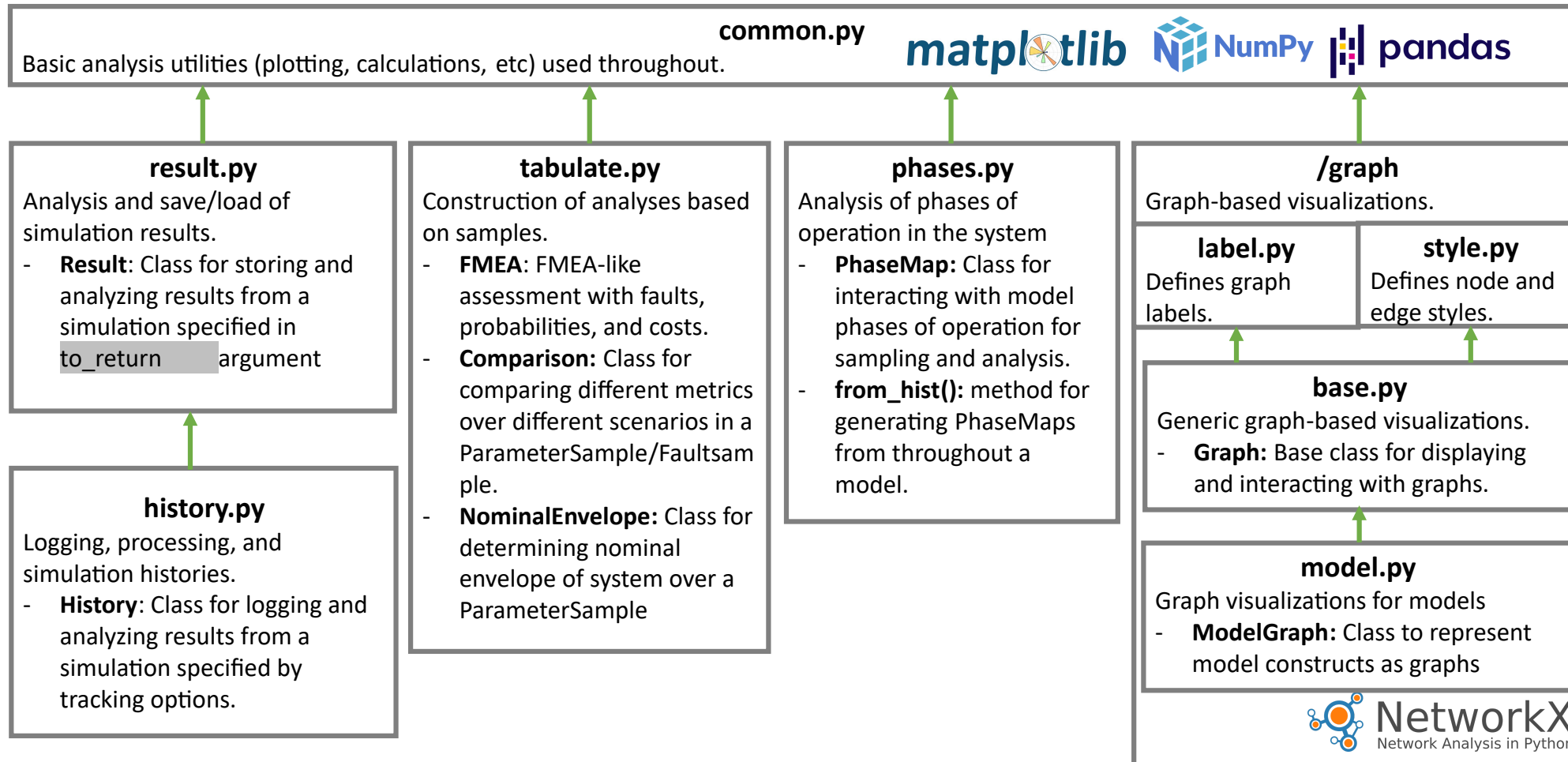
What do the results look like? Explore data structures:

- `analyze.result.Result` , `analyze.result.History`

Explore:

- What happens when you change `FaultSample` parameters?
- What happens when you change `Model` parameters?
- How do these methods compare in terms of computational time?

# Analysis Modules - see docs for `fmdtools.analyze`



# Analysis Activity

## Visualize the results:

- Show model graph
- Show nominal performances
- Show performances in a nominal scenario
- Make a scenario-based FMEA

## Explore:

- How can you show only the parameters you want? Or change the formatting?
- What does the behavior under other faults look like?
- What other analyses can you perform with these results?

# Starting your own repo/project

1.) Use `uv init` to create a template project, with its own virtual environment at `.venv`, `README.md`, and `pyproject.toml` files.

- Fill in these files (e.g., fmdtools version, contributors, project description)
- see: <https://docs.astral.sh/uv/guides/projects/>

2.) Install fmdtools into the venv:

- `uv pip install fmdtools` or `uv pip install fmdtools @ "../path/to/fmdtools"`

3.) Set up the rest of the python tooling: <https://github.com/nasa/fmdtools?tab=readme-ov-file#set-up-python-tooling-environment>

5.) See if it all works! Get a script to run fmdtools from the `venv` kernel in your IDE.

6.) Commit and sync to remote

# Conclusions/Summary

- **fmdtools** is an environment for designing resilient systems
  - `/define` enables model definition
  - `/sim` is used to define simulations
  - `/analyze` is used to analyze and visualize simulation results
- I hope you agree that it has some powerful features!
  - Modeling expressiveness and clarity
  - Types of simulations that can be run
  - Powerful but easy-to-leverage plug-and-play analyses

## Further Reading/Links

- More advanced topics ([see examples](#)), including Search and optimization, Human, Systems-of-Systems modeling, and Modeling Stochastic Behavior
- [Model Development Guide](#): Has best practices for developing models in a strategic way (especially helpful for complex models)
- Overview Paper:
  - [Hulse, D., Walsh, H., Dong, A., Hoyle, C., Tumer, I., Kulkarni, C., & Goebel, K. \(2021\). fmdtools: A fault propagation toolkit for resilience assessment in early design. International Journal of Prognostics and Health Management, 12\(3\).](#)